

Сравнение и анализ ассемблерного кода, сгенерированного исследуемыми компиляторами при различных уровнях оптимизации

Для каждого рассматриваемого метода оптимизации сначала приведён ассемблерный код, сгенерированный тремя wybranными компиляторами без оптимизаций (-O0) , а затем — с оптимизацией по скорости (-O2) . Это позволяет наглядно увидеть, какие преобразования применяет компилятор и как меняется структура кода.

Для удобства анализа из ассемблерных листингов удалены вызовы функции printf, а также инструкции, связанные с подготовкой её аргументов. Кроме того, исключены служебные инструкции пролога и эпилога функций (сохранение и восстановление регистров, выделение и освобождение места в стеке), а также директивы отладки .cfi_*. Оставлен только тот код, который непосредственно реализует вычисления, управляющие конструкции и обращения к памяти, необходимые для иллюстрации конкретной оптимизации.

Параметры сборки:

1) Без оптимизации:

- gcc -S -O0 optimization_test.c -o gcc_O0.s
- clang -S -O0 optimization_test.c -o clang_O0.s
- icc -S -O0 optimization_test.c -o icc_O0.s

2) С оптимизацией по скорости:

- gcc -S -O2 optimization_test.c -o gcc_O2.s
- clang -S -O2 optimization_test.c -o clang_O2.s
- icc -S -O2 optimization_test.c -o icc_O2.s

Размножение констант и копий

```
j4 = 2;
printf("%d", j4);
if (i2 < j4 && i4 < j4) {
    i2 = 2;
    printf("%d", i2);
}
j4 = k5;
printf("%d", j4);
if (i2 < j4 && i4 < j4) {
    i5 = 3;
    printf("%d", i5);
}
```

GCC	Clang	Intel C
<pre> movl \$2, j4(%rip) movl j4(%rip), %eax movl i2(%rip), %edx cmpl %eax, %edx jge .L2 movl i4(%rip), %edx movl j4(%rip), %eax cmpl %eax, %edx jge .L2 .L2: movl k5(%rip), %eax movl %eax, j4(%rip) movl j4(%rip), %eax movl i2(%rip), %edx cmpl %eax, %edx jge .L3 movl i4(%rip), %edx movl j4(%rip), %eax cmpl %eax, %edx jge .L3 </pre>	<pre> movl \$2, j4(%rip) movl i2(%rip), %eax cmpl j4(%rip), %eax jge .LBB0_3 # %bb.1: movl i4(%rip), %eax cmpl j4(%rip), %eax jge .LBB0_3 .LBB0_3: movl k5(%rip), %eax movl %eax, j4(%rip) movl i2(%rip), %eax cmpl j4(%rip), %eax jge .LBB0_6 # %bb.4: movl i4(%rip), %eax cmpl j4(%rip), %eax jge .LBB0_6 </pre>	<pre> movl \$2, j4 movl i2, %eax cmpl j4, %eax jge .LBB0_3 # %bb.1: movl i4, %eax cmpl j4, %eax jge .LBB0_3 .LBB0_3: movl k5, %eax movl %eax, j4 movl i2, %eax cmpl j4, %eax jge .LBB0_6 # %bb.4: movl i4, %eax cmpl j4, %eax jge .LBB0_6 </pre>

Ни один из компиляторов не демонстрирует явного применения размножения констант и размножения копий: значение j4 заново загружается из памяти при каждом сравнении, вместо использования известной константы 2 или непосредственной подстановки значения k5.

Clang и Intel C показали себя несколько лучше GCC, так как не стали помещать значение j4 в регистр для сравнения, а взяли его значение непосредственно из памяти, что сократило количество инструкций.

Отсутствие рассматриваемых оптимизаций в данном фрагменте можно объяснить тем, что переменная j4 является глобальной. Компилятор не может безопасно предположить, что её значение останется неизменным между моментом присваивания и последующим сравнением, и, как следствие, не выполняет размножение констант и копий.

GCC	Clang	Intel C
<pre> movl \$2, j4(%rip) movl j4(%rip), %eax cmpl %eax, i2(%rip) jge .L27 cmpl i4(%rip), %eax jg .L37 movl k5(%rip), %edx movl %edx, j4(%rip) movl j4(%rip), %eax cmpl %eax, i2(%rip) jge .L28 cmpl i4(%rip), %eax jg .L38 </pre>	<pre> movl \$2, j4(%rip) movl j4(%rip), %eax cmpl %eax, i2(%rip) jge .LBB0_3 cmpl %eax, i4(%rip) jge .LBB0_3 movl k5(%rip), %esi movl %esi, j4(%rip) movl j4(%rip), %eax cmpl %eax, i2(%rip) jge .LBB0_6 cmpl %eax, i4(%rip) jge .LBB0_6 </pre>	<pre> movl i2(%rip), %r14d movl i4(%rip), %ebp cmpl \$1, %r14d jg .LBB0_3 cmpl \$1, %ebp jg .LBB0_3 movl k5(%rip), %ebx cmpl %ebx, %r14d jge .LBB0_6 cmpl %ebx, %ebp jge .LBB0_6 </pre>

Компиляторы GCC и Clang не применили размножение констант: несмотря на присваивание $j4 = 2$, сравнения не преобразовались к виду с непосредственной константой. После присваивания $j4 = k5$ не было выполнено размножение копий — в сравнениях по-прежнему использовалась переменная $j4$.

Intel C, в свою очередь, применил эти оптимизации: вместо сравнения с $j4$ компилятор использует сравнение с константой: он преобразовал условие $i2 < 2$ в эквивалентное $i2 \leq 1$, а далее выполнил сравнение напрямую с переменной $k5$.

GCC и Clang придерживаются консервативного подхода к анализу глобальных переменных. Компиляторы ограничивают распространение констант и копий при недостатке информации о возможных побочных эффектах. Intel C же применяет более агрессивные оптимизации, поэтому заменяет обращение к переменной на непосредственное значение.

Свертка констант, арифметические тождества и излишние операции загрузки/сохранения

```
i3 = 1 + 2;
flt_1 = 2.4 + 6.3;
j2 = i + 0;
k2 = i / 1;
i4 = i * 1;
i5 = i * 0;
flt_3 = 2.4 / 1.0;
flt_4 = 1.0 + 0.0000001;
flt_5 = flt_6 * 0.0;
flt_6 = flt_2 * flt_3;
```

GCC	Clang	Intel C
<pre>movl \$3, i3(%rip) movsd .LC1(%rip), %xmm0 movsd %xmm0, flt_1(%rip) movl i(%rip), %eax movl %eax, j2(%rip) movl i(%rip), %eax movl %eax, k2(%rip) movl i(%rip), %eax movl %eax, i4(%rip) movl \$0, i5(%rip) movsd .LC3(%rip), %xmm0 movsd %xmm0, flt_3(%rip) movsd .LC4(%rip), %xmm0 movsd %xmm0, flt_4(%rip) movsd flt_6(%rip), %xmm1 pxor %xmm0, %xmm0 mulsd %xmm1, %xmm0 movsd %xmm0, flt_5(%rip) movsd flt_2(%rip), %xmm1 movsd flt_3(%rip), %xmm0 mulsd %xmm1, %xmm0</pre>	<pre>movl \$3, i3(%rip) movsd .LCPI0_2(%rip), %xmm0 movsd %xmm0, flt_1(%rip) movl i(%rip), %eax addl \$0, %eax movl %eax, j2(%rip) movl i(%rip), %eax movl \$1, %ecx cld idivl %ecx movl %eax, k2(%rip) movl i(%rip), %eax shll \$0, %eax movl %eax, i4(%rip) imull \$0, i(%rip), %eax movl %eax, i5(%rip) movsd .LCPI0_1(%rip), %xmm0 movsd %xmm0, flt_3(%rip) movsd .LCPI0_0(%rip),</pre>	<pre>movl \$3, i3 movsd .LCPI0_2(%rip), %xmm0 movsd %xmm0, flt_1 movl i, %eax addl \$0, %eax movl %eax, j2 movl i, %eax movl \$1, %ecx cld idivl %ecx movl %eax, k2 movl i, %eax shll \$0, %eax movl %eax, i4 imull \$0, i, %eax movl %eax, i5 movsd .LCPI0_1(%rip), %xmm0 movsd %xmm0, flt_3 movsd .LCPI0_0(%rip),</pre>

movsd %xmm0,flt_6(%rip)	%xmm0 movsd %xmm0,flt_4(%rip) xorps %xmm0,%xmm0 mulsd flt_6(%rip),%xmm0 movsd %xmm0,flt_5(%rip) movsd flt_2(%rip),%xmm0 mulsd flt_3(%rip),%xmm0 movsd %xmm0,flt_6(%rip)	%xmm0 movsd %xmm0,flt_4 xorps %xmm0,%xmm0 mulsd flt_6,%xmm0 movsd %xmm0,flt_5 movsd flt_2,%xmm0 mulsd flt_3,%xmm0 movsd %xmm0,flt_6
-------------------------	---	---

В данном фрагменте компиляторы выполняют свертку констант частично. Выражения вида $1 + 2$, $2.4 + 6.3$, $2.4 / 1.0$ и $1.0 + 0.0000001$ вычисляются на этапе компиляции и заменяются на готовые значения. Однако выражение $flt_6 * 0.0$ не упрощается до нуля, компиляторы генерируют инструкцию умножения. Это связано с особенностями стандарта IEEE 754: при участии значений NaN, бесконечности или ± 0 результат умножения на ноль может отличаться от нуля, поэтому такая оптимизация в общем случае небезопасна.

Однако поведение компиляторов различается при обработке арифметических тождеств. GCC корректно упрощает выражения $i + 0$, $i / 1$ и $i * 1$, заменяя их на простое копирование значения i , а выражение $i * 0$ — на присваивание нуля. Компиляторы Clang и Intel C не выполняют такие упрощения: в сгенерированном коде остаются операции сложения и умножения с нулем, а также операции деления и умножения на единицу.

Также все компиляторы повторно загружают значение переменной i из памяти для каждого присваивания, не храня его в регистре. Это связано с тем, что переменная является глобальной, и компиляторы не делают агрессивных предположений о ее неизменности.

Таким образом, наиболее эффективный код в данном фрагменте генерирует GCC, так как он наиболее полно выполняет алгебраические упрощения и устраняет лишние арифметические операции.

GCC	Clang	Intel C
movl \$3,i3(%rip) movq .LC1(%rip),%rax movq %rax,flt_1(%rip) movl i(%rip),%eax movl i(%rip),%eax movl %eax,j2(%rip) movl %eax,k2(%rip) movl %eax,i4(%rip) movl \$0,i5(%rip) movq .LC4(%rip),%rax movq %rax,flt_4(%rip) movsd .LC3(%rip),%xmm0 movsd %xmm0,flt_3(%rip) pxor %xmm1,%xmm1 mulsd flt_6(%rip),%xmm1 movsd %xmm1,flt_5(%rip) mulsd flt_2(%rip),%xmm0 movsd %xmm0,flt_6(%rip)	movl \$3,i3(%rip) movabsq \$4621087282649523814,%rax movq %rax,flt_1(%rip) movl i(%rip),%eax movl %eax,j2(%rip) movl %eax,k2(%rip) movl %eax,i4(%rip) movl \$0,i5(%rip) movabsq \$4612586738352862003,%rax movq %rax,flt_3(%rip) movabsq \$4607182419250377371,%rax movq %rax,flt_4(%rip) xorpd %xmm0,%xmm0	movl \$3,i3(%rip) movabsq \$4621087282649523814,%rax movq %rax,flt_1(%rip) movl i(%rip),%eax movl %eax,j2(%rip) movl %eax,k2(%rip) movl %eax,i4(%rip) movl \$0,i5(%rip) movabsq \$4612586738352862003,%rax movq %rax,flt_3(%rip) movabsq \$4607182419250377371,%rax movq %rax,flt_4(%rip) xorpd %xmm0,%xmm0

	mulsd flt_6(%rip), %xmm0 movsd %xmm0, flt_5(%rip) movsd flt_2(%rip), %xmm0 mulsd .LCPI0_0(%rip), %xmm0 movsd %xmm0, flt_6(%rip)	mulsd flt_6(%rip), %xmm0 movsd %xmm0, flt_5(%rip) movsd flt_2(%rip), %xmm0 mulsd .LCPI0_0(%rip), %xmm0 movsd %xmm0, flt_6(%rip)
--	--	--

В данном фрагменте все три компилятора выполняют свертку констант: целочисленное выражение $1 + 2$ заменяется на присваивание константы 3. Выражения с вещественными константами, такие как $2.4 + 6.3$, $2.4 / 1.0$ и $1.0 + 0.0000001$ также заменяются готовыми константными значениями. Реальная арифметическая инструкция, как и в случае без оптимизаций, генерируется только для вычисления $flt_6 * 0.0$ из-за особенностей стандарта IEEE 754.

Арифметические тождества для целых чисел упрощаются всеми компиляторами.

При этом качество итогового кода несколько различается. Clang и Intel C загружают значение i из памяти один раз и затем используют его для нескольких присваиваний, тогда как GCC выполняет повторную загрузку i , что приводит к появлению лишней инструкции.

Таким образом, по набору выполненных оптимизаций все три компилятора ведут себя сходным образом, однако Clang и Intel C генерируют немного более аккуратный код в части работы с целочисленной переменной i .

Лишнее присваивание

```
k3 = 1;
k3 = 1;
```

GCC	Clang	Intel C
movl \$1, k3(%rip) movl \$1, k3(%rip)	movl \$1, k3(%rip) movl \$1, k3(%rip)	movl \$1, k3 movl \$1, k3

В данном фрагменте ни один из компиляторов не устраняет лишнее повторное присваивание. В ассемблерном коде GCC, Clang и Intel C сохраняются две одинаковые инструкции записи значения 1 в переменную $k3$. Причина такого поведения, вероятно, связана с тем, что $k3$ является глобальной переменной. Для глобальных объектов компилятор не всегда может гарантировать отсутствие обращений извне.

GCC	Clang	Intel C
movl \$1, k3(%rip)	movl \$1, k3(%rip)	movl \$1, k3(%rip)

При уровне оптимизации -O2 все три компилятора устраняют лишнее повторное присваивание. Это означает, что компиляторы распознают избыточность последовательных записей одного и того же значения и не генерируют лишнюю инструкцию записи в итоговом коде.

Снижение мощности

```
k2 = 4 * j5;
```

```
for (i = 0; i <= 5; i++)
    ivector4[i] = i * 2;
```

GCC	Clang	Intel C
<pre> movl j5(%rip), %eax sall \$2, %eax movl %eax, k2(%rip) movl \$0, i(%rip) jmp .L4 .L5: movl i(%rip), %eax leal (%rax,%rax), %edx movl %edx, %ecx leaq ivector4(%rip), %rax movw %cx, (%rdx,%rax) movl i(%rip), %eax addl \$1, %eax movl %eax, i(%rip) .L4: movl i(%rip), %eax cmpl \$5, %eax jle .L5</pre>	<pre> movl j5(%rip), %eax shll \$2, %eax movl %eax, k2(%rip) movl \$0, i(%rip) .LBB0_7: cmpl \$5, i(%rip) jg .LBB0_10 # %bb.8: movl i(%rip), %eax shll %eax movw %ax, %dx movslq i(%rip), %rcx leaq ivector4(%rip), %rax movw %dx, (%rax, %rcx,2) # %bb.9: movl i(%rip), %eax addl \$1, %eax movl %eax, i(%rip) jmp .LBB0_7</pre>	<pre> movl j5, %eax shll \$2, %eax movl %eax, k2 movl \$0, i .LBB0_7: cmpl \$5, i jg .LBB0_10 # %bb.8: movl i, %eax shll %eax movw %ax, %cx movslq i, %rax movw %cx, ivector4(, %rax,2) # %bb.9: movl i, %eax addl \$1, %eax movl %eax, i jmp .LBB0_7</pre>

При вычислении $k2 = 4 * j5$ операция умножения всеми компиляторами заменяется на сдвиг влево на два бита, что эквивалентно умножению на 4. Аналогичное преобразование применяется и внутри цикла при вычислении $i * 2$. GCC реализует его как сложение $i + i$, тогда как Clang и Intel C используют сдвиг влево на один бит, оба подхода являются корректными вариантами снижения мощности.

GCC	Clang	Intel C
<pre> movl j5(%rip), %eax sall \$2, %eax movl %eax, k2(%rip) movq .LC6(%rip), %rax movq %rax, ivector4(%rip) movl \$655368, 8+ivector4(%rip)</pre>	<pre> movl j5(%rip), %eax shll \$2, %eax movl %eax, k2(%rip) movabsq \$1688867040264192, %rax movq %rax, ivector4(%rip) movl \$655368, ivector4+8(%rip)</pre>	<pre> movl j5(%rip), %eax shll \$2, %eax movl %eax, k2(%rip) movabsq \$1688867040264192, %rax movq %rax, ivector4(%rip) movl \$655368, ivector4+8(%rip)</pre>

В данном фрагменте все три компилятора также применяют снижение мощности: при вычислении $k2$ операция умножения заменяется на операцию сдвига.

Кроме того, на уровне оптимизации -O2 компиляторы полностью устраняют цикл. Поскольку число итераций известно заранее, а выражение $i * 2$ зависит только от переменной цикла, значения вычисляются на этапе компиляции. В результате цикл заменяется последовательностью присваиваний заранее вычисленных констант.

Простой цикл

```
j5 = 0;
k5 = 10000;
do {
    k5 = k5 - 1;
    j5 = j5 + 1;
    i5 = (k5 * 3) / (j5 * constant5);
} while (k5 > 0);
```

GCC	Clang	Intel C
<pre> movl \$0, j5(%rip) movl \$10000, k5(%rip) .L6: movl k5(%rip), %eax subl \$1, %eax movl %eax, k5(%rip) movl j5(%rip), %eax addl \$1, %eax movl %eax, j5(%rip) movl k5(%rip), %edx movl %edx, %eax addl %eax, %eax leal (%rax,%rdx), %ecx movl j5(%rip), %edx movl %edx, %eax sall \$2, %eax leal (%rdx,%rax), %esi movl %ecx, %eax cld idivl %esi movl %eax, i5(%rip) movl k5(%rip), %eax testl %eax, %eax jg .L6 </pre>	<pre> movl \$0, j5(%rip) movl \$10000, k5(%rip) .LBB0_11: movl k5(%rip), %eax subl \$1, %eax movl %eax, k5 movl j5, %eax addl \$1, %eax movl %eax, j5 imull \$3, k5, %eax imull \$5, j5, %ecx cld idivl %ecx movl %eax, i5(%rip) # %bb.12: cmpl \$0, k5(%rip) jg .LBB0_11 </pre>	<pre> movl \$0, j5 movl \$10000, k5 .LBB0_11: movl k5, %eax subl \$1, %eax movl %eax, k5 movl j5, %eax addl \$1, %eax movl %eax, j5 imull \$3, k5, %eax imull \$5, j5, %ecx cld idivl %ecx movl %eax, i5 # %bb.12: cmpl \$0, k5 jg .LBB0_11 </pre>

Все три компилятора на уровне оптимизации -O0 генерируют полный цикл. Цикл выполняется в явном виде, переменные k5 и j5 загружаются и сохраняются на каждой итерации, а выражение с делением вычисляется полностью, без упрощений.

Основное различие между компиляторами проявляется в реализации умножения на константы. GCC заменил операции умножения на 3 и 5 на последовательности сложений и сдвигов. А компиляторы Clang и Intel C использовали прямые инструкции умножения с константами.

GCC	Clang	Intel C
movl \$10000, j5(%rip) movl \$0, i5(%rip)	movl \$10000, j5(%rip) movl \$0, i5(%rip)	movl \$10000, j5(%rip) movl \$0, i5(%rip)

Компиляторы полностью оптимизировали цикл: вместо последовательного выполнения 10000 итераций они вычислили итоговый эффект цикла на этапе компиляции и заменили его прямыми присваиваниями. Такое преобразование возможно потому, что цикл имеет фиксированное число итераций и влияет только на конечные значения переменных, то есть промежуточные значения не важны.

Управление переменной индукции цикла

```
for (i = 0; i < 100; i++)
    ivector5[i * 2 + 3] = 5;
```

GCC	Clang	Intel C
.L8: movl \$0, i(%rip) movl i(%rip), %eax addl %eax, %eax addl \$3, %eax leaq 0(,%rax,4), %rdx leaq ivector5(%rip), %rax movl \$5, (%rdx,%rax) movl i(%rip), %eax addl \$1, %eax movl %eax, i(%rip) .L7: movl i(%rip), %eax cmpl \$99, %eax jle .L8	# %bb.13: movl \$0, i(%rip) .LBB0_14: cmpl \$100, i(%rip) jge .LBB0_17 # %bb.15: movl i(%rip), %eax shll %eax addl \$3, %eax movslq %eax, %rcx leaq ivector5(%rip), %rax movl \$5, (%rax,%rcx,4) # %bb.16: movl i(%rip), %eax addl \$1, %eax movl %eax, i(%rip) jmp .LBB0_14	# %bb.13: movl \$0, i .LBB0_14: cmpl \$100, i jge .LBB0_17 # %bb.15: movl i, %eax shll %eax addl \$3, %eax movl \$5, ivector5(%rax,4) # %bb.16: movl i, %eax addl \$1, %eax movl %eax, i jmp .LBB0_14

Переменная индукции цикла не удалена ни одним из рассматриваемых компиляторов. Исходная переменная *i* сохраняется в памяти и на каждой итерации загружается снова, после чего с ее помощью вычисляется индекс $i * 2 + 3$, а затем обновленное значение *i* записывается обратно.

Здесь можно отметить локальное применение снижения мощности. Выражение $i * 2$ во всех компиляторах заменяется на более простую операцию: GCC реализует его как сложение $i + i$, тогда как Clang и Intel C используют сдвиг влево на один бит.

GCC	Clang	Intel C
leaq 12+ivector5(%rip), %rax	leaq ivector5(%rip), %rcx .LBB0_7:	movq \$-768, %rax .LBB0_7:

<pre> leaq 800(%rax), %rdx .L29: movl \$5, (%rax) addq \$16, %rax movl \$5, -8(%rax) cmpq %rax, %rdx jne .L29 </pre>	<pre> movl \$5, 12(%rcx,%rax,8) movl \$5, 20(%rcx,%rax,8) movl \$5, 28(%rcx,%rax,8) movl \$5, 36(%rcx,%rax,8) addq \$4, %rax cmpq \$100, %rax jne .LBB0_7 </pre>	<pre> movl \$5, ivector5+780(%rax) movl \$5, ivector5+788(%rax) movl \$5, ivector5+796(%rax) movl \$5, ivector5+804(%rax) movl \$5, ivector5+812(%rax) movl \$5, ivector5+820(%rax) movl \$5, ivector5+828(%rax) movl \$5, ivector5+836(%rax) addq \$64, %rax jne .LBB0_7 </pre>
--	--	--

Все три компилятора применили оптимизацию удаления переменной индукции цикла. Обращение к элементам массива реализуется через адресную арифметику. Условие выхода из цикла также преобразуется и выражается через сравнение адресов.

Дополнительно во всех трёх компиляторах применяется разворачивание цикла различной степени агрессивности.

Глубокие подвыражения

```

if (i < 10)
    j5 = i5 + i2;
else
    k5 = i5 + i2;

```

GCC	Clang	Intel C
<pre> movl i(%rip), %eax cmpl \$9, %eax jg .L9 movl i5(%rip), %edx movl i2(%rip), %eax addl %edx, %eax movl %eax, j5(%rip) jmp .L10 .L9: movl i5(%rip), %edx movl i2(%rip), %eax addl %edx, %eax movl %eax, k5(%rip) </pre>	<pre> .LBB0_17: cmpl \$10, i(%rip) jge .LBB0_19 # %bb.18: movl i5(%rip), %eax addl i2(%rip), %eax movl %eax, j5(%rip) jmp .LBB0_20 .LBB0_19: movl i5(%rip), %eax addl i2(%rip), %eax movl %eax, k5(%rip) </pre>	<pre> .LBB0_17: cmpl \$10, i jge .LBB0_19 # %bb.18: movl i5, %eax addl i2, %eax movl %eax, j5 jmp .LBB0_20 .LBB0_19: movl i5, %eax addl i2, %eax movl %eax, k5 </pre>

Во всех трёх случаях выражение $i5 + i2$ вычисляется отдельно в каждой ветви условного оператора, а не выносится за его пределы, то есть устранение общего подвыражения здесь не выполняется.

GCC	Clang	Intel C
<pre>movl i2(%rip), %eax movl %eax, k5(%rip)</pre>	<pre>movl i2(%rip), %eax movl %eax, k5(%rip)</pre>	<pre>movl \$5, k5(%rip)</pre>

Все компиляторы выполнили удаление недостижимого кода. Поскольку переменной i ранее было присвоено значение 100, условие $i < 10$ всегда ложно, и ветка `if` никогда не будет выполнена. В результате компиляторы полностью устранили ее, оставив только ветку `else`.

Помимо этого, в этом фрагменте выполняется упрощение выражения $i5 + i2$. Так как переменная $i5$ имеет значение 0, значение $k5$ не вычисляется с помощью сложения, а сводится к присваиванию ей значения переменной $i2$, что является примером использования арифметического тождества.

GCC и Clang генерируют код, в котором значение $i2$ загружается из памяти и копируется в $k5$. Intel C в данном фрагменте также применяет констант, заменяя выражение на непосредственное значение 5, полностью устраняя обращение к памяти.

Генерация адреса переменной с константным индексом, размножение копий и регистров

```
ivector[0] = 1;
printf("%d", ivector[0]);
ivector[i2] = 2;
printf("%d", ivector[i2]);
ivector[i2] = 2;
printf("%d", ivector[i2]);
ivector[2] = 3;
printf("%d", ivector[2]);
```

GCC	Clang	Intel C
<pre>movl \$1, ivector(%rip) movl i2(%rip), %eax leaq 0(,%rax,4), %rdx leaq ivector(%rip), %rax movl \$2, (%rdx,%rax) movl i2(%rip), %eax leaq 0(,%rax,4), %rdx leaq ivector(%rip), %rax movl \$2, (%rdx,%rax) movl i2(%rip), %eax movl \$3, 8+ivector(%rip)</pre>	<pre>movl \$1, ivector(%rip) movslq i2(%rip), %rcx leaq ivector(%rip), %rax movl \$2, (%rax,%rcx,4) movslq i2(%rip), %rcx leaq ivector(%rip), %rax movl \$2, (%rax,%rcx,4) movl \$3, ivector+8(%rip) movl ivector+8(%rip), %esi</pre>	<pre>movl \$1, ivector movslq i2, %rax movl \$2, ivector(,%rax,4) movslq i2, %rax movl \$2, ivector(,%rax,4) movl \$3, ivector+8 movl ivector+8, %esi</pre>

Генерация адресов элементов массива с константным индексом выполняется через фиксированные смещения относительно базового адреса массива.

При обращении к элементам массива с переменным индексом `i2` компиляторы используют различные способы вычисления адреса. GCC вычисляет адрес поэтапно: сначала значение индекса загружается в регистр, затем умножается на размер элемента массива, после чего к результату прибавляется базовый адрес массива. Clang использует более компактную форму адресации, в которой вычисление адреса выполняется непосредственно в инструкции через выражение вида «база + индекс × масштаб», что позволяет избежать промежуточных вычислений. При этом ни один из этих двух компиляторов не оптимизирует использование регистров: значение переменной `i2` загружается из памяти перед каждым использованием, базовый адрес массива также не сохраняется в регистре. Intel C применяет более агрессивный подход: значение индекса определяется на этапе компиляции, и обращение к массиву сводится к использованию фиксированного смещения относительно базового адреса. В результате полностью устраняется необходимость в вычислении адреса на этапе выполнения.

GCC	Clang	Intel C
<pre>leaq ivector(%rip), %rbp movl \$1, ivector(%rip) movslq i2(%rip), %rax movl \$2, 0(%rbp,%rax,4) movslq i2(%rip), %rax movl \$2, 0(%rbp,%rax,4) movl \$3, 8+ivector(%rip)</pre>	<pre>movl \$1, ivector(%rip) movslq i2(%rip), %rax movl \$2, (%r14,%rax,4) movslq i2(%rip), %rax movl \$2, (%r14,%rax,4) movl \$3, ivector+8(%rip)</pre>	<pre>movl \$1, ivector(%rip) movl \$2, ivector+20(%rip) movl \$3, ivector+8(%rip)</pre>

Все компиляторы используют адресацию элементов массива через смещения. Для константных индексов адрес формируется как базовый адрес массива плюс фиксированное смещение. При использовании переменного индекса GCC и Clang применяют адресацию вида «база + индекс × масштаб». А Intel C заменяет переменную `i2` её известным значением. После этого компилятор вычисляет адрес как фиксированное смещение (`ivector+20`, так как `i2 = 5`).

В компиляторах GCC и Clang генерируются две отдельные инструкции записи для двух одинаковых присваиваний `ivector[i2] = 2`, при этом значение переменной `i2` по-прежнему загружается из памяти перед каждым использованием, повторное использование регистров для переменной `i2` не наблюдается. Intel C, в свою очередь, удаляет избыточное повторное присваивание, что является примером удаления мёртвого кода.

Таким образом, Intel C демонстрирует более глубокую оптимизацию, объединяя размножение констант, устранение лишних операций и сокращение обращений к памяти, тогда как GCC и Clang ограничиваются базовой адресацией и не применяют более агрессивные преобразования.

Удаление общих подвыражений

```
if ((h3 + k3) < 0 || (h3 + k3) > 5)
    printf("Common subexpression elimination\n");
else {
    m3 = (h3 + k3) / i3;
    printf("%d", m3);
    g3 = i3 + (h3 + k3);
    printf("%d", g3);
}
```

GCC	Clang	Intel C
<pre> movl h3(%rip), %edx movl k3(%rip), %eax addl %edx, %eax testl %eax, %eax js .L11 movl h3(%rip), %edx movl k3(%rip), %eax addl %edx, %eax cmpl \$5, %eax jle .L12 .L11: jmp .L13 .L12: movl h3(%rip), %edx movl k3(%rip), %eax addl %edx, %eax movl i3(%rip), %ecx idivl %ecx movl %eax, m3(%rip) movl k3(%rip), %eax addl %eax, %edx movl i3(%rip), %eax addl %edx, %eax movl %eax, g3(%rip) movl g3(%rip), %eax </pre>	<pre> movl h3(%rip), %eax addl k3(%rip), %eax cmpl \$0, %eax jl .LBB0_22 # %bb.21: movl h3(%rip), %eax addl k3(%rip), %eax cmpl \$5, %eax jle .LBB0_23 .LBB0_22: jmp .LBB0_24 .LBB0_23: movl h3(%rip), %eax addl k3(%rip), %eax idivl i3(%rip) movl %eax, m3(%rip) movl i3(%rip), %eax movl h3(%rip), %ecx addl k3(%rip), %ecx addl %ecx, %eax movl %eax, g3(%rip) </pre>	<pre> movl h3, %eax addl k3, %eax cmpl \$0, %eax jl .LBB0_22 # %bb.21: movl h3, %eax addl k3, %eax cmpl \$5, %eax jle .LBB0_23 .LBB0_22: jmp .LBB0_24 .LBB0_23: movl h3, %eax addl k3, %eax idivl i3 movl %eax, m3 movl i3, %eax movl h3, %ecx addl k3, %ecx addl %ecx, %eax movl %eax, g3 </pre>

Сумма $h3 + k3$ вычисляется многократно: перед каждым сравнением в условии и повторно внутри ветки else. Во всех трёх компиляторах это выражение каждый раз вычисляется заново путём загрузки операндов из памяти и выполнения операции сложения. Таким образом, удаление общего подвыражения не выполняется: компиляторы не сохраняют результат вычисления и не переиспользуют его, что приводит к дублированию одинаковых вычислений.

GCC	Clang	Intel C
<pre> movl k3(%rip), %eax addl h3(%rip), %eax cmpl \$5, %eax jbe .L30 idivl i3(%rip) movl %eax, m3(%rip) movl k3(%rip), %edx addl h3(%rip), %edx addl i3(%rip), %edx movl %edx, g3(%rip) </pre>	<pre> movl k3(%rip), %eax addl h3(%rip), %eax cmpl \$6, %eax jb .LBB0_10 idivl i3(%rip) movl %eax, m3(%rip) movl h3(%rip), %esi addl i3(%rip), %esi addl k3(%rip), %esi movl %esi, g3(%rip) </pre>	<pre> movl h3(%rip), %ebx leal 1(%rbx), %eax cmpl \$5, %eax jbe .LBB0_9 movl \$2863311531, %esi imulq %rax, %rsi shrq \$33, %rsi movl %esi, m3(%rip) addl \$4, %ebx movl %ebx, g3(%rip) </pre>

На уровне оптимизации -O2 компиляторы GCC и Clang выполняют частичное устранение общего подвыражения $h3 + k3$. Сумма вычисляется один раз и используется при проверке условия и для вычисления значения $m3 = (h3 + k3) / i3$. Однако после выполнения операции деления значение в регистре перезаписывается, но исходная сумма не сохраняется в отдельном регистре. В результате при вычислении $g3 = i3 + (h3 + k3)$ выражение $h3 + k3$ вычисляется повторно, что указывает на отсутствие полного устранения общего подвыражения.

Компилятор Intel C выполняет распространение констант, подставляя значения $k3 = 1$ и $i3 = 3$, после чего выражения упрощаются. Деление на константу заменяется на умножение на заранее вычисленную константу с последующим сдвигом, а выражение $g3 = i3 + (h3 + k3)$ сводится к $h3 + 4$. Таким образом, необходимость в явном устранении общего подвыражения исчезает за счёт более глубокого преобразования выражений.

Во всех компиляторах наблюдается упрощение условного выражения: две проверки объединяются в одну с использованием беззнакового сравнения, что уменьшает количество переходов.

Вынесение инвариантного кода

```
for (i4 = 0; i4 <= max_vector; i4++){
    ivector2[i4] = j * k;
    printf("%d", ivector2[i4]);
}
```

GCC	Clang	Intel C
<pre>.L13: movl \$0, i4(%rip) jmp .L14 .L15: movl j(%rip), %eax movl %eax, %esi movl k(%rip), %eax movl %eax, %edx movl i4(%rip), %ecx movl %esi, %eax imull %edx, %eax movl %eax, %edx movslq %ecx, %rax leaq ivector2(%rip), %rcx movb %dl, (%rax,%rcx) movl i4(%rip), %eax addl \$1, %eax movl %eax, i4(%rip) .L14: movl i4(%rip), %eax cmpl \$2, %eax jle .L15</pre>	<pre>.LBB0_24: movl \$0, i4(%rip) .LBB0_25: cmpl \$2, i4(%rip) jg .LBB0_28 # %bb.26: movl j(%rip), %eax imull k(%rip), %eax movb %al, %dl movslq i4(%rip), %rcx leaq ivector2(%rip), %rax movb %dl, (%rax,%rcx) # %bb.27: movl i4(%rip), %eax addl \$1, %eax movl %eax, i4(%rip) jmp .LBB0_25</pre>	<pre>.LBB0_24: movl \$0, i4 .LBB0_25: cmpl \$2, i4 jg .LBB0_28 # %bb.26: movl j, %eax imull k, %eax movb %al, %cl movslq i4, %rax movb %cl, ivector2(, %rax) # %bb.27: movl i4, %eax addl \$1, %eax movl %eax, i4 jmp .LBB0_25</pre>

Ни один из компиляторов не выполняет вынесение инвариантного кода из цикла. В сгенерированном коде на каждой итерации заново выполняются загрузка переменных *j* и *k* из памяти и их последующее умножение, хотя выражение *j * k* не зависит от переменной цикла *i4* и потому могло бы быть вычислено один раз перед входом в цикл.

GCC	Clang	Intel C
<pre> .L32 movzbl j(%rip), %eax movslq %ecx, %rcx mulb k(%rip) movb %al, 0(%rb,%rcx) movl i4(%rip), %eax leal 1(%rax), %ecx movl %ecx, i4(%rip) cmpl \$2, %ecx jle .L32 </pre>	<pre> .LBB0_12: movl k(%rip), %ecx imull j(%rip), %ecx cltq movb %cl, (%rax,%r14) movl i4(%rip), %ecx leal 1(%rcx), %eax movl %eax, i4(%rip) cmpl \$2, %ecx jl .LBB0_12 </pre>	<pre> .LBB0_10: movl k(%rip), %eax imull j(%rip), %eax movzbl %al, %ebx xorl %r14d, %r14d .LBB0_11: movb %bl, ivector2(%r14) leaq 1(%r14), %r15 movl %r15d, i4(%rip) cmpl \$2, %r14d jge .LBB0_15 # %bb.12: movb %bl, ivector2+1(%r14) leal 2(%r14), %eax movl %eax, i4(%rip) cmpl \$1, %r15d jg .LBB0_15 # %bb.13: movb %bl, ivector2+2(%r14) leal 3(%r14), %eax movl %eax, i4(%rip) cmpl \$2147483646, %r14d jb .LBB0_15 </pre>

GCC и Clang не выполнили оптимизацию вынесения инвариантного кода даже при уровне оптимизации -O2: в ассемблерном коде, сгенерированном этими компиляторами сохраняется загрузка *j* и *k* из памяти и их умножение на каждой итерации. Вероятно, это связано с ограничениями анализа глобальных переменных.

Intel C, напротив, вычисляет произведение *j * k* один раз до основного тела цикла, после чего использует готовое значение внутри тела цикла. Кроме того, Intel C применил развёртывание цикла. Поскольку число итераций известно заранее, компилятор заменил цикл последовательностью нескольких однотипных операций записи.

Вызов функции с аргументами

```
dead_code(1, "This line should not be printed");
```

GCC	Clang	Intel C
call dead_code	callq dead_code	callq dead_code

Вызов функции `dead_code` в `main` сохраняется у всех трёх компиляторов. Это означает, что компиляторы не выполняют удаление бесполезного вызова функции.

GCC	Clang	Intel C
Инструкция <code>call dead_code</code> отсутствует в <code>main</code> .	Инструкция <code>call dead_code</code> отсутствует в <code>main</code> .	Инструкция <code>call dead_code</code> отсутствует в <code>main</code> .

При уровне оптимизации `-O2` все три компилятора полностью удаляют вызов функции `dead_code` из `main`. Это происходит, потому что данная функция не влияет на поведение программы: условие `if (0)` недостижимо, а присваивание локальной переменной не используется.

Удаление недостижимого кода

```
void dead_code(int a, char *b){
    int idead_store;
    idead_store = a;
    if (0)
        printf("%s\n", b);
}
```

GCC	Clang	Intel C
<code>dead_code:</code> <code>.LFB1:</code> <code>movl %edi,</code> <code>-20(%rbp)</code> <code>movl -20(%rbp),</code> <code>%eax</code> <code>movl %eax, -4(%rbp)</code> <code>ret</code>	<code>dead_code:</code> <code># %bb.0:</code> <code>movl %edi, -4(%rbp)</code> <code>movl -4(%rbp), %eax</code> <code>movl %eax,</code> <code>-20(%rbp)</code> <code>retq</code>	<code>dead_code:</code> <code># %bb.0:</code> <code>movl %edi, -4(%rbp)</code> <code>movl -4(%rbp), %eax</code> <code>movl %eax,</code> <code>-20(%rbp)</code> <code>retq</code>

В данном фрагменте все три компилятора выполняют удаление недостижимого кода: ветка `if (0)` распознаётся как заведомо невыполнимая, поэтому вызов `printf` внутри функции `dead_code` в ассемблерном коде отсутствует.

Однако присваивание `idead_store = a` остаётся в коде у всех трёх компиляторов. Значение локальной переменной `idead_store` далее нигде не используется и такая запись является бесполезной, однако компиляторы не удаляют мертвое присваивание.

GCC	Clang	Intel C
<code>dead_code:</code> <code>.LFB38:</code> <code>ret</code>	<code>dead_code:</code> <code># %bb.0:</code> <code>retq</code>	<code>dead_code:</code> <code># %bb.0:</code> <code>retq</code>

При уровне оптимизации -O2 все три компилятора полностью устраняют тело функции `dead_code`, оставляя только инструкцию возврата. Это означает, что были применены оптимизации удаления мертвого кода и мертвого присваивания.

Вызов функции без аргументов

`unnecessary_loop();`

GCC	Clang	Intel C
<code>call unnecessary_loop</code>	<code>call unnecessary_loop</code>	<code>callq unnecessary_loop</code>

Вызов функции `unnecessary_loop` в `main` сохраняется у всех трёх компиляторов.

GCC	Clang	Intel C
<code>call unnecessary_loop</code>	Инструкция <code>call unnecessary_loop</code> отсутствует в <code>main</code> .	Инструкция <code>call unnecessary_loop</code> отсутствует в <code>main</code> .

Clang и Intel C выполняют более агрессивную межпроцедурную оптимизацию: после упрощения тела функции `unnecessary_loop` необходимость в отдельном вызове функции исчезает, поэтому инструкция `call` удаляется из `main`, заменяя вызов оптимизированным телом функции. GCC в данном случае сохраняет отдельный вызов функции.

Удаление бесполезного цикла

```
void unnecessary_loop(){
    int x;
    x = 0;
    for (i = 0; i < 5; i++)
        k5 = x + j5;
    printf("%d", k5);
}
```

GCC	Clang	Intel C
<code>unnecessary_loop:</code> <code>.LFB2:</code> <code>movl \$0, -4(%rbp)</code> <code>movl \$0, i(%rip)</code> <code>jmp .L19</code> <code>.L20:</code> <code>movl j5(%rip), %edx</code> <code>movl -4(%rbp), %eax</code> <code>addl %edx, %eax</code> <code>movl %eax, k5(%rip)</code> <code>movl i(%rip), %eax</code> <code>addl \$1, %eax</code> <code>movl %eax, i(%rip)</code> <code>.L19:</code> <code>movl i(%rip), %eax</code>	<code>unnecessary_loop:</code> <code># %bb.0:</code> <code>movl \$0, -4(%rbp)</code> <code>movl \$0, i(%rip)</code> <code>.LBB2_1:</code> <code>cmpl \$5, i(%rip)</code> <code>jge .LBB2_4</code> <code># %bb.2:</code> <code>movl -4(%rbp), %eax</code> <code>addl j5(%rip), %eax</code> <code>movl %eax, k5(%rip)</code> <code># %bb.3:</code> <code>movl i(%rip), %eax</code> <code>addl \$1, %eax</code> <code>movl %eax, i(%rip)</code>	<code>unnecessary_loop:</code> <code># %bb.0:</code> <code>movl \$0, -4(%rbp)</code> <code>movl \$0, i</code> <code>.LBB2_1:</code> <code>cmpl \$5, i</code> <code>jge .LBB2_4</code> <code># %bb.2:</code> <code>movl -4(%rbp), %eax</code> <code>addl j5, %eax</code> <code>movl %eax, k5</code> <code># %bb.3:</code> <code>movl i, %eax</code> <code>addl \$1, %eax</code> <code>movl %eax, i</code>

cmpl \$4, %eax jle .L20 ret	jmp .LBB2_1 .LBB2_4: retq	jmp .LBB2_1 .LBB2_4: retq
-----------------------------------	---------------------------------	---------------------------------

Ни один из компиляторов не выполняет удаление бесполезного цикла. Несмотря на то, что переменная x равна нулю и выражение в теле цикла сводится к $k5 = j5$, функция `unnecessary_loop` содержит все исходные операции, хотя цикл мог быть заменён одним присваиванием.

GCC	Clang	Intel C
unnecessary_loop: .LFB39: movl j5(%rip), %edx movl %edx, k5(%rip)	unnecessary_loop: # %bb.0: movl j5(%rip), %esi movl %esi, k5(%rip)	unnecessary_loop: # %bb.0: movl j5(%rip), %esi movl %esi, k5(%rip)

При компиляции с флагом `-O2` бесполезный цикл был удален всеми рассматриваемыми компиляторами: выражение в теле цикла сводится к $k5 = j5$.

Слияние циклов

```

loop_jamming(7);
void loop_jamming(int x){
    for (i = 0; i < 5; i++) {
        k5 = x + j5 * i;
        printf("%d", k5);
    }
    for (i = 0; i < 5; i++) {
        i5 = x * k5 * i;
        printf("%d", k5);
    }
}

```

GCC	Clang	Intel C
loop_jamming: .LFB3: movl %edi, -4(%rbp) movl \$0, i(%rip) jmp .L22 .L23: movl j5(%rip), %edx movl i(%rip), %eax imull %eax, %edx movl -4(%rbp), %eax addl %edx, %eax movl %eax, k5(%rip) movl i(%rip), %eax addl \$1, %eax movl %eax, i(%rip)	loop_jamming: # %bb.0: movl %edi, -4(%rbp) movl \$0, i(%rip) .LBB3_1: cmpl \$5, i(%rip) jge .LBB3_4 # %bb.2: movl -4(%rbp), %eax movl j5(%rip), %ecx imull i(%rip), %ecx addl %ecx, %eax movl %eax, k5(%rip) # %bb.3: movl i(%rip), %eax	loop_jamming: # %bb.0: movl %edi, -4(%rbp) movl \$0, i .LBB3_1: cmpl \$5, i jge .LBB3_4 # %bb.2: movl -4(%rbp), %eax movl j5, %ecx imull i, %ecx addl %ecx, %eax movl %eax, k5 # %bb.3: movl i, %eax

.L22: movl i(%rip), %eax cmpl \$4, %eax jle .L23 movl \$0, i(%rip) jmp .L24 .L25: movl k5(%rip), %eax imull -4(%rbp), %eax movl %eax, %edx movl i(%rip), %eax imull %edx, %eax movl %eax, i5(%rip) movl i(%rip), %eax addl \$1, %eax movl %eax, i(%rip) .L24: movl i(%rip), %eax cmpl \$4, %eax jle .L25 ret	addl \$1, %eax movl %eax, i(%rip) jmp .LBB3_1 .LBB3_4: movl \$0, i(%rip) .LBB3_5: cmpl \$5, i(%rip) jge .LBB3_8 # %bb.6: movl -4(%rbp), %eax imull k5(%rip), %eax imull i(%rip), %eax movl %eax, i5(%rip) # %bb.7: movl i(%rip), %eax addl \$1, %eax movl %eax, i(%rip) jmp .LBB3_5 .LBB3_8: retq	addl \$1, %eax movl %eax, i jmp .LBB3_1 .LBB3_4: movl \$0, i .LBB3_5: cmpl \$5, i jge .LBB3_8 # %bb.6: movl -4(%rbp), %eax imull k5, %eax imull i, %eax movl %eax, i5 # %bb.7: movl i, %eax addl \$1, %eax movl %eax, i jmp .LBB3_5 .LBB3_8: retq
--	--	--

Все три компилятора сохраняют оба цикла в исходном виде: сначала полностью выполняется первый цикл, затем заново инициализируется счётчик *i* и выполняется второй цикл. Это означает, что слияние циклов не выполняется. Такое поведение логично, поскольку второй цикл зависит от результата первого: в первом цикле вычисляется и обновляется переменная *k5*, а во втором цикле ее итоговое значение используется в вычислении $i5 = x * k5 * i$. Следовательно, итерации двух циклов не являются независимыми и не могут быть объединены в один цикл.

GCC	Clang	Intel C
loop_jamming: .LFB40: movl %edi, %ebx movl \$0, i(%rip) .L5: imull j5(%rip), %edx addl %ebx, %edx movl %edx, k5(%rip) movl i(%rip), %eax leal 1(%rax), %edx movl %edx, i(%rip) cmpl \$4, %edx jle .L5 movl \$0, i(%rip) .L6: movl k5(%rip), %ecx movl %ecx, %edx imull %ebx, %edx imull %edx, %eax	loop_jamming: # %bb.0: movl %edi, %ebx movl \$0, i(%rip) .LBB3_1: imull j5(%rip), %esi addl %ebx, %esi movl %esi, k5(%rip) movl i(%rip), %eax leal 1(%rax), %esi movl %esi, i(%rip) cmpl \$4, %eax jl .LBB3_1 # %bb.2: movl \$0, i(%rip) .LBB3_3: movl k5(%rip), %esi imull %ebx, %eax imull %esi, %eax	loop_jamming: # %bb.0: movl %edi, %ebx movl \$0, i(%rip) movl j5(%rip), %r14d movl %edi, k5(%rip) leal (%rbx,%r14), %esi movl %esi, k5(%rip) leal (%rbx,%r14,2), %esi movl %esi, k5(%rip) leal (%r14,%r14,2), %esi addl %ebx, %esi movl %esi, k5(%rip) leal (%rbx,%r14,4), %ebp

<pre> movl %eax, i5(%rip) movl i(%rip), %eax addl \$1, %eax movl %eax, i(%rip) cmpl \$4, %eax jle .L6 </pre>	<pre> movl %eax, i5(%rip) movq %r14, %rdi movl i(%rip), %ecx leal 1(%rcx), %eax movl %eax, i(%rip) cmpl \$4, %ecx jl .LBB3_3 </pre>	<pre> movl %ebp, k5(%rip) movl \$0, i5(%rip) movl \$1, i(%rip) imull %ebp, %ebx movl %ebx, i5(%rip) movl \$2, i(%rip) leal (%rbx,%rbx), %eax movl %eax, i5(%rip) movl \$3, i(%rip) leal (%rbx,%rbx,2), %eax movl %eax, i5(%rip) movl \$4, i(%rip) shll \$2, %ebx movl %ebx, i5(%rip) </pre>
--	---	--

Слияние циклов не выполнено. GCC и Clang сохраняют два отдельных цикла: после завершения первого цикла переменная *i* заново инициализируется, и только затем начинается выполнение второго. Intel C разворачивает оба цикла, заменяя их на последовательность вычислений и присваиваний.

Отсутствие слияния циклов в данном случае свидетельствует о корректном анализе зависимостей между циклами, так как второй цикл зависит от результата первого. Таким образом, все три компилятора корректно отказываются от слияния, а Intel C применяет иную оптимизацию.

Разворачивание циклов

```

loop_unrolling(7);
void loop_unrolling(int x){
    for (i = 0; i < 6; i++) {
        ivector4[i] = 0;
        printf("%d", ivector4[i]);
    }
}

```

GCC	Clang	Intel C
<pre> loop_unrolling: .LFB4: movl %edi, -4(%rbp) movl \$0, i(%rip) jmp .L27 .L28: movl i(%rip), %eax cltq leaq (%rax,%rax), %rdx leaq ivector4(%rip), </pre>	<pre> loop_unrolling: # %bb.0: movl %edi, -4(%rbp) movl \$0, i(%rip) .LBB4_1: cmpl \$6, i(%rip) jge .LBB4_4 # %bb.2: movslq i(%rip), %rcx leaq ivector4(%rip), %rax </pre>	<pre> loop_unrolling: # %bb.0: movl %edi, -4(%rbp) movl \$0, i .LBB4_1: cmpl \$6, i jge .LBB4_4 # %bb.2: movslq i, %rax movw \$0, ivector4(, %rax,2) </pre>

<pre> %rax movw \$0, (%rdx,%rax) movl i(%rip), %eax addl \$1, %eax movl %eax, i(%rip) .L27: movl i(%rip), %eax cmpl \$5, %eax jle .L28 ret </pre>	<pre> movw \$0, (%rax,%rcx,2) # %bb.3: movl i(%rip), %eax addl \$1, %eax movl %eax, i(%rip) jmp .LBB4_1 .LBB4_4: retq </pre>	<pre> # %bb.3: movl i, %eax addl \$1, %eax movl %eax, i jmp .LBB4_1 .LBB4_4: retq </pre>
---	--	--

Разворачивание цикла не выполнено ни одним из компиляторов. В ассемблерном коде GCC, Clang и Intel C сохраняется обычный цикл со счётчиком i, проверкой условия выхода и переходом к следующей итерации.

GCC	Clang	Intel C
<pre> loop_unrolling: .LFB41: leaq ivector4(%rip), %rbp movl \$0, i(%rip) .L11: movw %dx, 0(%rbp, %rax,2) movl i(%rip), %eax addl \$1, %eax movl %eax, i(%rip) cmpl \$5, %eax jle .L11 </pre>	<pre> loop_unrolling: # %bb.0: movl \$0, i(%rip) leaq ivector4(%rip), %r14 .LBB4_1: movw \$0, (%r14,%rax,2) movl i(%rip), %ecx leal 1(%rcx), %eax movl %eax, i(%rip) cmpl \$5, %ecx jl .LBB4_1 </pre>	<pre> loop_unrolling: # %bb.0: movq \$0, ivector4(%rip) movl \$4, i(%rip) movw \$0, ivector4+10(%rip) movl \$5, i(%rip) movw \$0, ivector4+10(%rip) </pre>

Intel C – единственный компилятор, который в данном случае применил разворачивание цикла. Компилятор сгенерировал последовательность операций записи в массив ivector4, устраняя переходы. GCC и Clang сохранили исходную циклическую структуру.

Сжатие цепочки переходов

```

jump_compression(1, 2, 3, 4, 5);
int jump_compression(int i, int j, int k, int l, int m){
beg_1:
    if (i < j)
        if (j < k)
            if (k < l)
                if (l < m)
                    l += m;
                else
                    goto end_1;
            else
                k += l;
        else{
            j += k;

```

```

        printf("%d", j);
    end_1:
        goto beg_1;
    }
else
    i += j;
return(i + j + k + l + m);
}

```

GCC	Clang	Intel C
jump_compression: .LFB5: movl %edi, -4(%rbp) movl %esi, -8(%rbp) movl %edx, -12(%rbp) movl %ecx, -16(%rbp) movl %r8d, -20(%rbp) .L30: movl -4(%rbp), %eax cmpl -8(%rbp), %eax jge .L31 movl -8(%rbp), %eax cmpl -12(%rbp), %eax jge .L32 movl -12(%rbp), %eax cmpl -16(%rbp), %eax jge .L33 movl -16(%rbp), %eax cmpl -20(%rbp), %eax jge .L38 movl -20(%rbp), %eax addl %eax, -16(%rbp) jmp .L35 .L33: movl -16(%rbp), %eax addl %eax, -12(%rbp) jmp .L35 .L32: movl -12(%rbp),	jump_compression: # %bb.0: movl %edi, -4(%rbp) movl %esi, -8(%rbp) movl %edx, -12(%rbp) movl %ecx, -16(%rbp) movl %r8d, -20(%rbp) .LBB5_1: movl -4(%rbp), %eax cmpl -8(%rbp), %eax jge .LBB5_13 # %bb.2: movl -8(%rbp), %eax cmpl -12(%rbp), %eax jge .LBB5_10 # %bb.3: movl -12(%rbp), %eax cmpl -16(%rbp), %eax jge .LBB5_8 # %bb.4: movl -16(%rbp), %eax cmpl -20(%rbp), %eax jge .LBB5_6 # %bb.5: movl -20(%rbp), %eax addl -16(%rbp), %eax movl %eax, -16(%rbp) jmp .LBB5_7 .LBB5_6: jmp .LBB5_11	jump_compression: # %bb.0: movl %edi, -4(%rbp) movl %esi, -8(%rbp) movl %edx, -12(%rbp) movl %ecx, -16(%rbp) movl %r8d, -20(%rbp) .LBB5_1: movl -4(%rbp), %eax cmpl -8(%rbp), %eax jge .LBB5_13 # %bb.2: movl -8(%rbp), %eax cmpl -12(%rbp), %eax jge .LBB5_10 # %bb.3: movl -12(%rbp), %eax cmpl -16(%rbp), %eax jge .LBB5_8 # %bb.4: movl -16(%rbp), %eax cmpl -20(%rbp), %eax jge .LBB5_6 # %bb.5: movl -20(%rbp), %eax addl -16(%rbp), %eax movl %eax, -16(%rbp) jmp .LBB5_7 .LBB5_6: jmp .LBB5_11

<pre> %eax .L38: addl %eax, -8(%rbp) jmp .L30 nop .L36: jmp .L30 .L31: movl -8(%rbp), %eax addl %eax, -4(%rbp) .L35: movl -4(%rbp), %edx movl -8(%rbp), %eax addl %eax, %edx movl -12(%rbp), %eax addl %eax, %edx movl -16(%rbp), %eax addl %eax, %edx movl -20(%rbp), %eax addl %edx, %eax ret </pre>	<pre> .LBB5_7: jmp .LBB5_9 .LBB5_8: movl -16(%rbp), %eax addl -12(%rbp), %eax movl %eax, -12(%rbp) .LBB5_9: jmp .LBB5_12 .LBB5_10: movl -12(%rbp), %eax addl -8(%rbp), %eax movl %eax, -8(%rbp) .LBB5_11: jmp .LBB5_1 .LBB5_12: jmp .LBB5_14 .LBB5_13: movl -8(%rbp), %eax addl -4(%rbp), %eax movl %eax, -4(%rbp) .LBB5_14: movl -4(%rbp), %eax addl -8(%rbp), %eax addl -12(%rbp), %eax addl -16(%rbp), %eax addl -20(%rbp), %eax retq </pre>	<pre> .LBB5_7: jmp .LBB5_9 .LBB5_8: movl -16(%rbp), %eax addl -12(%rbp), %eax movl %eax, -12(%rbp) .LBB5_9: jmp .LBB5_12 .LBB5_10: movl -12(%rbp), %eax addl -8(%rbp), %eax movl %eax, -8(%rbp) .LBB5_11: jmp .LBB5_1 .LBB5_12: jmp .LBB5_14 .LBB5_13: movl -8(%rbp), %eax addl -4(%rbp), %eax movl %eax, -4(%rbp) .LBB5_14: movl -4(%rbp), %eax addl -8(%rbp), %eax addl -12(%rbp), %eax addl -16(%rbp), %eax addl -20(%rbp), %eax retq </pre>
--	---	---

В данном ассемблерном листинге при уровне оптимизации -O0 ни один из компиляторов не выполняет сжатие цепочки переходов. Сгенерированный код точно сохраняет структуру вложенных условий исходной программы. Каждая проверка оформляется отдельным блоком, а управление передается через последовательность меток и переходов.

GCC	Clang	Intel C
<pre> jump_compression: .LFB42: cmpl %edi, %esi jg .L15 jmp .L16 .L25: cmpl %r13d, %ebp jge .L18 cmpl %r14d, %r13d </pre>	<pre> jump_compression: # %bb.0: cmpl %edi, %esi jle .LBB5_9 # %bb.1: jmp .LBB5_2 .LBB5_7: addl %r14d, %ebx .LBB5_8: </pre>	<pre> jump_compression: # %bb.0: cmpl %esi, %edi jl .LBB5_1 .LBB5_8: addl %r14d, %r12d .LBB5_9: addl %r15d, %r14d addl %r12d, %r14d </pre>

.L15:	jl .L24	.LBB5_2:	cmpl %r15d, %ebx	.LBB5_6:	addl %ebp, %ebx
	cmpl %r12d, %ebx		jle .LBB5_9		addl %r14d, %ebx
	jle .L16		cmpl %r14d, %ebx		movl %ebx, %eax
	cmpl %ebx, %ebp		jge .LBB5_7		addl %r15d, %r14d
.L16:	jg .L25	# %bb.3:	cmpl %r12d, %r14d	.LBB5_7:	cmpl %r14d, %r12d
	addl %ebp, %ebx		jge .LBB5_6		jge .LBB5_8
	cmpl %r12d, %ebx		cmpl %ebp, %r12d		.LBB5_1:
	jg .L15		jge .LBB5_8		cmpl %r15d, %r14d
.L20:	addl %ebx, %r12d	# %bb.5:	addl %ebp, %r12d	# %bb.2:	jge .LBB5_6
	leal (%r12,%rbx), %eax		jmp .LBB5_10		cmpl %ebx, %r15d
	addl %ebp, %eax		.LBB5_9:		jge .LBB5_5
	addl %r13d, %eax		addl %ebx, %r15d	# %bb.3:	cmpl %ebp, %ebx
.L18:	addl %r14d, %eax	.LBB5_10:	addl %r12d, %r14d		jge .LBB5_7
	addl %r13d, %ebp		addl %ebp, %ebx		.LBB5_4:
	jmp .L20		addl %r14d, %ebx		addl %ebp, %ebx
	addl %r14d, %r13d		addl %r15d, %ebx	.LBB5_5:	jmp .LBB5_9
.L24:	jmp .L20		movl %ebx, %eax		addl %ebx, %r15d
					jmp .LBB5_9

Исходная глубокая вложенность условий перестраивается всеми анализируемыми компиляторами в более плоскую структуру. При этом часть промежуточных переходов устраняется. В отличие от неоптимизированного кода, где управление происходит через последовательность промежуточных меток, в оптимизированной версии такие цепочки сокращаются, и выполнение направляются сразу в конечные блоки. Это делает поток управления более компактным и уменьшает количество переходов.